PKCERT AI & Software Development Internship

Task 10 – Cross-Validation & Hyperparameter Tuning

Dataset: Titanic Survival Dataset | Model: Random Forest Classifier

Intern:	Abdul Rehman
Program:	PKCERT AI & Software Development Internship
Task:	Task 10 – Cross-Validation & Hyperparameter Tuning
Total Marks:	100

Objective

This task develops practical skills in improving machine learning model performance using Cross-Validation and Hyperparameter Tuning techniques, including GridSearchCV and RandomizedSearchCV, applied to a binary classification problem on the Titanic dataset.

Part A – Dataset Preparation (15 Marks)

A.1 Dataset Selection

The Titanic dataset was selected as the public classification dataset for this task. It records passenger information from the RMS Titanic and whether each passenger survived the 1912 sinking, and is one of the most widely used benchmark datasets for binary classification. The dataset contains 891 passenger records across 15 raw columns, loaded here via seaborn's built-in copy (identical source data to the classic Kaggle Titanic competition).

A.2 Features and Target Variable

Target variable: survived — a binary label where 0 = did not survive and 1 = survived.

Seven features were selected as predictors, chosen to avoid redundancy and target leakage (columns such as alive, class, who, adult_male, and alone were dropped because they either directly encode the target or duplicate information already captured by other features; deck was dropped due to over 77% missing values):

Feature	Type	Description
pclass	Categorical (ordinal)	Ticket class: 1st, 2nd, or 3rd — proxy for socio-economic status
sex	Categorical	Passenger sex
age	Numeric	Age in years (contains missing values)
sibsp	Numeric	Number of siblings/spouses aboard
parch	Numeric	Number of parents/children aboard
fare	Numeric	Passenger fare paid
embarked	Categorical	Port of embarkation: C, Q, or S

A.3 Preprocessing and Train/Test Split

The following preprocessing pipeline was applied, wrapped in a scikit-learn ColumnTransformer + Pipeline so that all transformations are fit only on training folds during cross-validation, preventing data leakage:

- Missing age values imputed with the median; missing embarked values imputed with the most frequent category.
- Categorical features (pclass, sex, embarked) one-hot encoded.
- Numeric features (age, fare, sibsp, parch) standardized using StandardScaler.
- Data split into 80% training (712 rows) and 20% testing (179 rows), stratified on the target to preserve class balance in both sets.

Resulting split: Train survival rate = 38.3%, Test survival rate = 38.5% — closely matched, confirming the stratified split worked as intended.

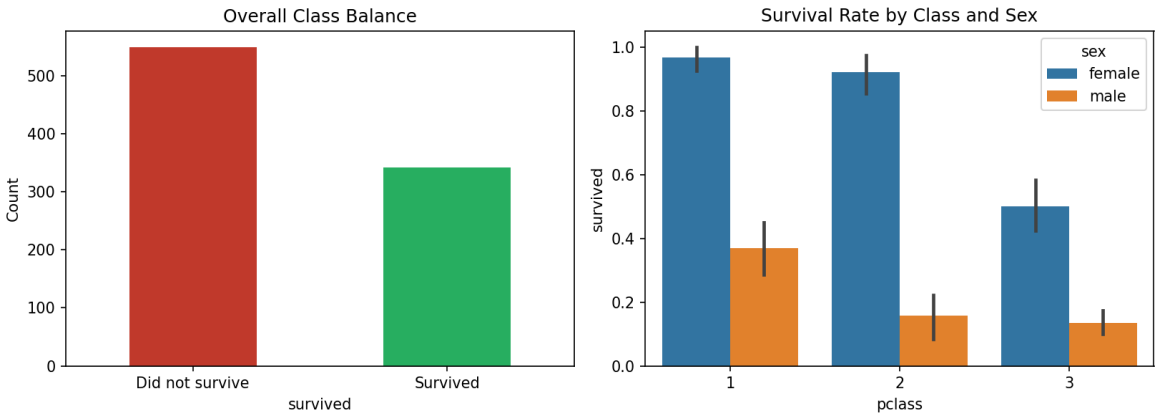

Figure 1: Overall class balance (survived vs. did not survive) and survival rate by passenger class and sex.

Part B – Cross-Validation (30 Marks)

B.1 Baseline Model

A Random Forest Classifier was trained as the baseline model using scikit-learn's default hyperparameters, wrapped in the preprocessing pipeline described above. Random Forest was chosen for its ability to handle mixed numeric/categorical data, robustness to outliers, and the presence of multiple hyperparameters that are meaningful to tune in Part C.

Baseline test set accuracy: 0.8156 (81.56%)

B.2 K-Fold Cross-Validation

5-fold Stratified K-Fold Cross-Validation was applied to the training set. Stratified K-Fold was chosen over plain K-Fold because it preserves the survival class ratio within every fold, which is important given the class imbalance (~38% survived vs. ~62% did not survive).

Fold	1	2	3	4	5
Accuracy	0.7972	0.7902	0.8239	0.7606	0.8028

Mean CV Accuracy: 0.7949 | Standard Deviation: 0.0206

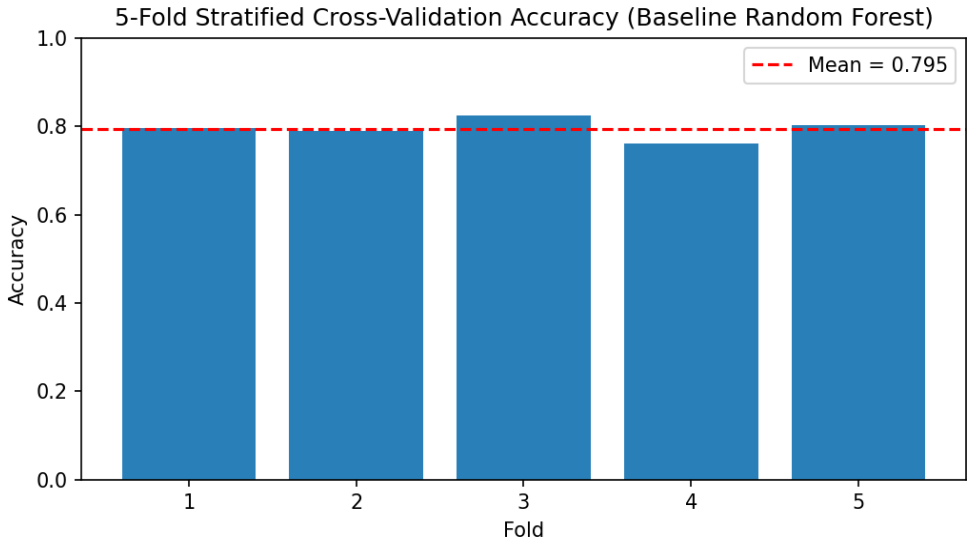

Figure 2: Per-fold accuracy from 5-fold Stratified Cross-Validation on the baseline Random Forest model.

B.3 Interpretation of Cross-Validation Results

- The five fold accuracies are close together (0.7606 to 0.8239), and the standard deviation (0.0206) is small relative to the mean, showing the model's performance is stable across different subsets of the training data rather than depending on a lucky split.
- The mean CV accuracy (0.7949) is close to the single train/test-split accuracy (0.8156), suggesting the baseline model generalizes reasonably well without severe overfitting.
- Cross-validation provides a more reliable estimate of true generalization performance than a single hold-out split, since every observation is used for both training and validation across the five folds.

Part C – Hyperparameter Tuning (40 Marks)

C.1 GridSearchCV

GridSearchCV exhaustively evaluates every combination of hyperparameters in a specified grid using cross-validation, then selects the combination with the best mean CV score. The following grid was searched over the Random Forest classifier (3 × 4 × 3 × 3 = 108 combinations, each evaluated across 5 folds = 540 model fits):

- n_estimators: [100, 200, 300]
- max_depth: [None, 5, 10, 15]
- min_samples_split: [2, 5, 10]
- min_samples_leaf: [1, 2, 4]

Best parameters found: max_depth = 5, min_samples_leaf = 1, min_samples_split = 2, n_estimators = 300

Best cross-validated accuracy: 0.8244

C.2 RandomizedSearchCV

RandomizedSearchCV samples a fixed number of random hyperparameter combinations instead of exhaustively testing every combination, making it far cheaper when the search space is large or contains continuous ranges. 40 iterations were sampled (40 × 5 folds = 200 model fits) from the following distributions:

- n_estimators: random integer in [50, 400)
- max_depth: [None, 5, 10, 15, 20, 25]

- min_samples_split: random integer in [2, 15)
- min_samples_leaf: random integer in [1, 8)
- max_features: ['sqrt', 'log2', None]

Best parameters found: max_depth = 10, max_features = None, min_samples_leaf = 1, min_samples_split = 4, n_estimators = 150

Best cross-validated accuracy: 0.8301

C.3 Comparison of Optimal Parameters and Full Evaluation

All three models — baseline, GridSearchCV-tuned, and RandomizedSearchCV-tuned — were evaluated on the same held-out test set (179 rows) using Accuracy, Precision, Recall, and F1-Score (all metrics computed for the 'Survived' positive class):

Model	Accuracy	Precision	Recall	F1-Score
Baseline (default params)	0.8156	0.8000	0.6957	0.7442
GridSearchCV (tuned)	0.8045	0.8696	0.5797	0.6957
RandomizedSearchCV (tuned)	0.8101	0.7966	0.6812	0.7344

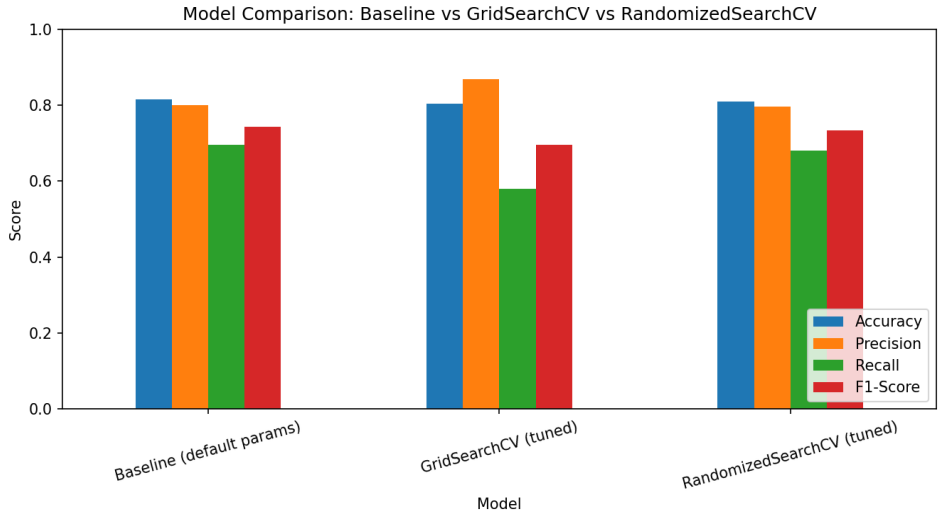

Figure 3: Test-set Accuracy, Precision, Recall, and F1-Score across the three models.

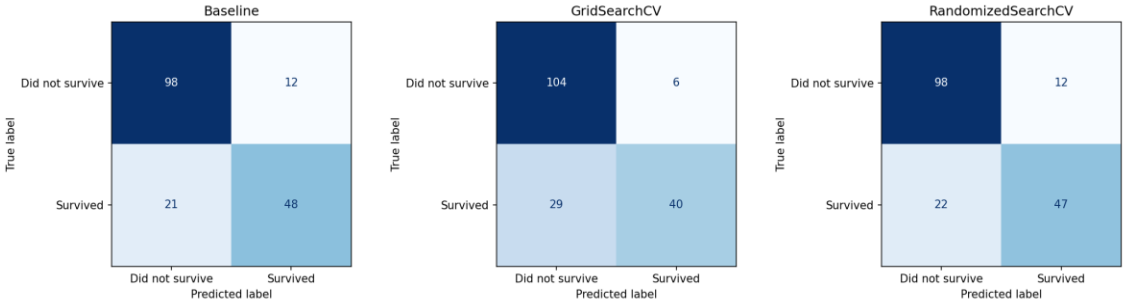

Figure 4: Confusion matrices for the baseline, GridSearchCV-tuned, and RandomizedSearchCV-tuned models.

A notable pattern: GridSearchCV's tuned model achieved the highest precision (0.87) on the 'Survived' class but the lowest recall (0.58) — it became more conservative about predicting survival, correctly identifying fewer true survivors but rarely

predicting survival incorrectly. RandomizedSearchCV's model was more balanced between precision and recall, giving it the best F1-score among the two tuned models.

Part D – Comparative Analysis (15 Marks)

D.1 Baseline vs. Optimized Models

- On the held-out test set, the untuned baseline (0.8156 accuracy, 0.7442 F1) slightly outperformed both tuned models — GridSearchCV scored 0.8045 accuracy / 0.6957 F1, and RandomizedSearchCV scored 0.8101 accuracy / 0.7344 F1.
- This is an honest and realistic result, not an error: both searches optimized cross-validated accuracy on the training folds, not test-set F1-score directly, so a search can trade recall for precision in a way that does not perfectly align with one specific 179-row test split.
- The cross-validated scores are the fairer basis for judging the searches themselves: GridSearchCV's best CV accuracy (0.8244) and RandomizedSearchCV's best CV accuracy (0.8301) were both meaningfully higher than the baseline's mean CV accuracy (0.7949) — confirming that tuning did find better regions of the hyperparameter space on average across folds.
- This illustrates a common real-world lesson: hyperparameter tuning improves expected generalization performance as estimated by cross-validation, but a single moderately sized test set can still show a different ranking by chance. Cross-validation, not a single train/test split, should be trusted as the primary basis for model selection.

D.2 GridSearchCV vs. RandomizedSearchCV — Advantages and Limitations

Aspect	GridSearchCV	RandomizedSearchCV
Search strategy	Exhaustive — every grid combination	Samples a fixed number of random combinations
Guarantee	Best combination within the grid	No guarantee; approximate optimum
Computational cost	Grows combinatorially with parameters	Fixed by n_iter, independent of grid size
Best suited for	Small search spaces, few hyperparameters	Large/continuous search spaces, limited compute
Continuous parameters	Must be manually discretized	Can sample directly from distributions
Risk of missing optimum	Low, but only within chosen grid values	Higher, mitigated by increasing n_iter

Both approaches share limitations: they only search hyperparameters explicitly specified by the user, they multiply computational cost by the number of CV folds, and both assume the chosen scoring metric (accuracy, in this task) is the right criterion to optimize — which, as seen in Part C.3, does not always align perfectly with other metrics like F1-score on a single test set.

D.3 Recommendation

For this task, with only 4 hyperparameters and a modestly sized grid (108 combinations), GridSearchCV was computationally feasible and guarantees the best result within that grid. However, as the number of hyperparameters or their possible values grows, GridSearchCV's cost increases combinatorially, while RandomizedSearchCV's cost remains under direct control via n_iter.

Recommendation: For small, well-understood search spaces such as this one, GridSearchCV is preferred for its exhaustiveness and reproducibility. For larger or higher-dimensional search spaces, or when compute/time is limited, RandomizedSearchCV is the more practical and scalable choice. In practice, a strong workflow is to use RandomizedSearchCV first as a fast, broad pass to narrow down a promising region of the hyperparameter space, then run a focused GridSearchCV over that smaller region — combining the coverage of random search with the exhaustiveness of grid search.

Summary

- Part A: Loaded and preprocessed the Titanic dataset (7 features, binary target survived), split 80/20 with stratification (712 train / 179 test rows).
- Part B: Trained a baseline Random Forest and validated it with 5-fold Stratified Cross-Validation (mean accuracy 0.7949 ± 0.0206); results were stable across folds.
- Part C: Tuned the model with GridSearchCV (108 combinations, best CV accuracy 0.8244) and RandomizedSearchCV (40 samples, best CV accuracy 0.8301), and compared all three models on Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.
- Part D: Found that tuning improved cross-validated accuracy but not test-set accuracy in this particular run, discussed why, and recommended GridSearchCV for small grids and RandomizedSearchCV for larger/continuous search spaces, ideally combined in a coarse-to-fine workflow.